

Java Backend Interview Guide (3–5 Years Experience)

Focus: Real-world Decision Making & Spring Boot Internals

1. Why does a Spring Boot app consume more memory over time?

This is often due to **metaspace growth** from dynamic class loading, **leaking ThreadLocals**, or unclosed resources (like DB connections). It can also be caused by **unbounded caching** or the JVM not returning memory to the OS quickly.

2. How do you detect bean initialization issues in large applications?

Use the **-debug flag** or `ConditionEvaluationReport` to see why beans were (or weren't) created. For circular dependencies or slow starts, **Spring Beans Graph** in IDEs or custom `BeanPostProcessors` can help track timing.

3. What happens if `@PostConstruct` throws an exception?

If an exception is thrown, the **application context fails to refresh**, and the JVM will shut down. This is because the bean is considered "broken" during its **mandatory lifecycle setup**, halting the entire startup process.

4. Why does `@Value` sometimes fail to inject properties?

This usually happens if the bean is created **manually via new** (bypassing Spring), or if `PropertySourcesPlaceholderConfigurer` isn't static. It can also fail if the property is **overridden by an environment variable** with a typo.

5. How does Spring Boot decide the order of auto-configurations?

Spring uses the **@AutoConfigureOrder**, **@AutoConfigureAfter**, or **@AutoConfigureBefore** annotations. Internally, it reads the **AutoConfiguration imports** file and sorts them based on these explicit hints or alphabetical order.

6. What are the risks of enabling too many Actuator endpoints?

Enabling too many (like `/heapdump` or `/env`) poses **security risks** by exposing sensitive credentials or system paths. Performance-wise, some endpoints can cause **significant CPU/Memory spikes** if polled too frequently by monitoring tools.

7. Why does your app behave differently after scaling pods?

Different behavior usually stems from **stateful logic** (storing data in local memory instead of Redis) or **race conditions**. You might also see **database connection pool exhaustion** as each new pod demands its own set of connections.

8. How does Spring Boot handle classpath scanning internally?

Spring uses the `ClassPathBeanDefinitionScanner` to crawl directories. It uses **ASM (a bytecode library)** to read class metadata without actually loading the classes into the JVM, which **saves memory** during the startup phase.

9. What causes duplicate bean registration in multi-module projects?

This happens when **multiple @ComponentScan configs** overlap or when a library includes its own `@Configuration` that isn't filtered. It often occurs in multi-module setups when a "common" module is scanned **redundantly**.

10. Why does your API return correct data but response time fluctuates?

This is often caused by **Stop-the-World GC pauses**, "cold" JIT compilation, or **downstream latency** in a microservice. It could also be **thread pool saturation** where requests wait in a queue before being processed.

11. How do you control thread usage in Spring Boot applications?

You manage this by configuring the **Tomcat/Undertow thread pool** and using `@Async` with a **custom ThreadPoolTaskExecutor**. Proper sizing involves

balancing the number of threads against the available **CPU cores** and **I/O wait times**.

12. What happens when **application.yml** and **application.properties** both exist?

If both exist in the same location, **application.properties** takes precedence over **application.yml**. Spring Boot loads both, but the properties file will **overwrite** any duplicate keys found in the YAML file.

13. Why do custom exception handlers sometimes not trigger?

This happens if the exception is thrown **outside the DispatcherServlet scope** (like in a Filter or Interceptor). It can also fail if a **more specific @ExceptionHandler** exists in another @ControllerAdvice and takes priority.

14. How do you handle large payloads without killing performance?

Use **Streaming (InputStream)** instead of mapping the whole body to a String/POJO to keep memory low. Implement **Jackson's incremental parsing** and ensure your reverse proxy (Nginx) doesn't have restrictive **buffer limits**.

15. Why does Hibernate generate unexpected queries?

This is usually the **N+1 problem** caused by FetchType.LAZY on collections without join-fetching. It can also be **"Open Session in View"** causing queries to run during the rendering phase outside your service logic.

16. How do you debug a deadlock in Spring?

Use jstack or jcmd to get a **Thread Dump** and look for the "BLOCKED" state. Analyze the **Lock Owner ID** to see which thread holds the monitor; tools like VisualVM visualize these **circular wait** dependencies.

17. What happens if a BeanFactoryPostProcessor fails?

Since these run **before bean instantiation**, a failure here stops the application immediately. It prevents the context from even knowing how to build your beans, essentially **killing the app at the blueprint stage**.

18. How do you avoid startup failure due to missing configs?

Use **@RequiredProperty** or default values in `@Value("${prop:default}")`. You can also use **EnvironmentPostProcessor** to validate essential keys early and provide "fail-fast" logs that are actually readable.

19. Why does Spring Boot retry DB connections on startup?

Spring Boot (via HikariCP) retries to ensure **resiliency against transient network blips**. If the DB is just slow to start (like in Docker Compose), retries prevent the app from **crashing immediately** on a temporary "connection refused."

20. How do you manage feature toggles safely?

Use a dedicated library like **Togglz** or **Unleash**. Avoid hardcoded if/else blocks by using **Configuration Composition** or Conditional beans (`@ConditionalOnProperty`) to swap implementations without logic clutter.

21. Why does @Cacheable sometimes not cache?

This usually happens due to **Self-Invocation** (calling the method from within the same class), which bypasses the Spring Proxy. It can also fail if the **CacheManager** is **misconfigured** or if the method returns null.

22. How does Spring Boot isolate environment-specific configs?

Spring uses **Profiles (spring.profiles.active)**. This allows loading specific application-`{profile}.yml` files. For sensitive data, use **Environment Variables** which override any file-based configuration by default.

23. What causes classloader issues in fat JARs?

Spring Boot uses a **custom LaunchedURLClassLoader** to read nested JARs. Conflicts arise when multiple versions of a library exist on the path, causing **NoSuchMethodError** because the wrong version was loaded first.

24. How do you safely reload configs without restarting?

Use **Spring Cloud Bus** or the `@RefreshScope` annotation with Actuator's `/refresh` endpoint. This re-instantiates beans with the **new configuration values** fetched from a Config Server or updated environment.

25. Why does logging behave differently in prod vs local?

Production usually uses **JSON formatting** for ELK/Splunk and "INFO" level, while local uses **ANSI colors** and "DEBUG". This is managed via `logback-spring.xml` using `<springProfile>` tags to switch appenders.

26. How do you handle partial failures in dependent services?

Implement the **Circuit Breaker pattern** (Resilience4j) to prevent cascading failures. Use **Fallbacks** to return cached or default data, ensuring the entire system doesn't hang because one non-essential service is down.

27. What is the real impact of using too many interceptors?

Each interceptor adds **latency to the request lifecycle**. If they perform blocking I/O (like auth checks or DB hits), they can **exhaust the request threads** and significantly degrade the overall throughput of the API.

28. How do you prevent breaking changes during deployments?

Use **API Versioning** (`/v1/`, `/v2/`) and Contract Testing (Pact). Ensure your DB migrations (Flyway/Liquibase) are **backward compatible** (e.g., add columns, don't delete them) to support "Blue-Green" deployments.

29. Why does @ConfigurationProperties fail silently?

This occurs if **getters/setters are missing** (required for data binding) or if the prefix is wrong. Unlike @Value, it doesn't always throw an exception on startup unless you use **@Validated** to enforce constraints.

30. What Spring Boot decision has caused you a real production issue?

A common one is **Default Connection Pool Sizing** being too small for high-concurrency bursts, leading to ConnectionTimeoutException. Another is **Circular Dependencies** masked by allow-circular-references: true, making the dependency graph complex.